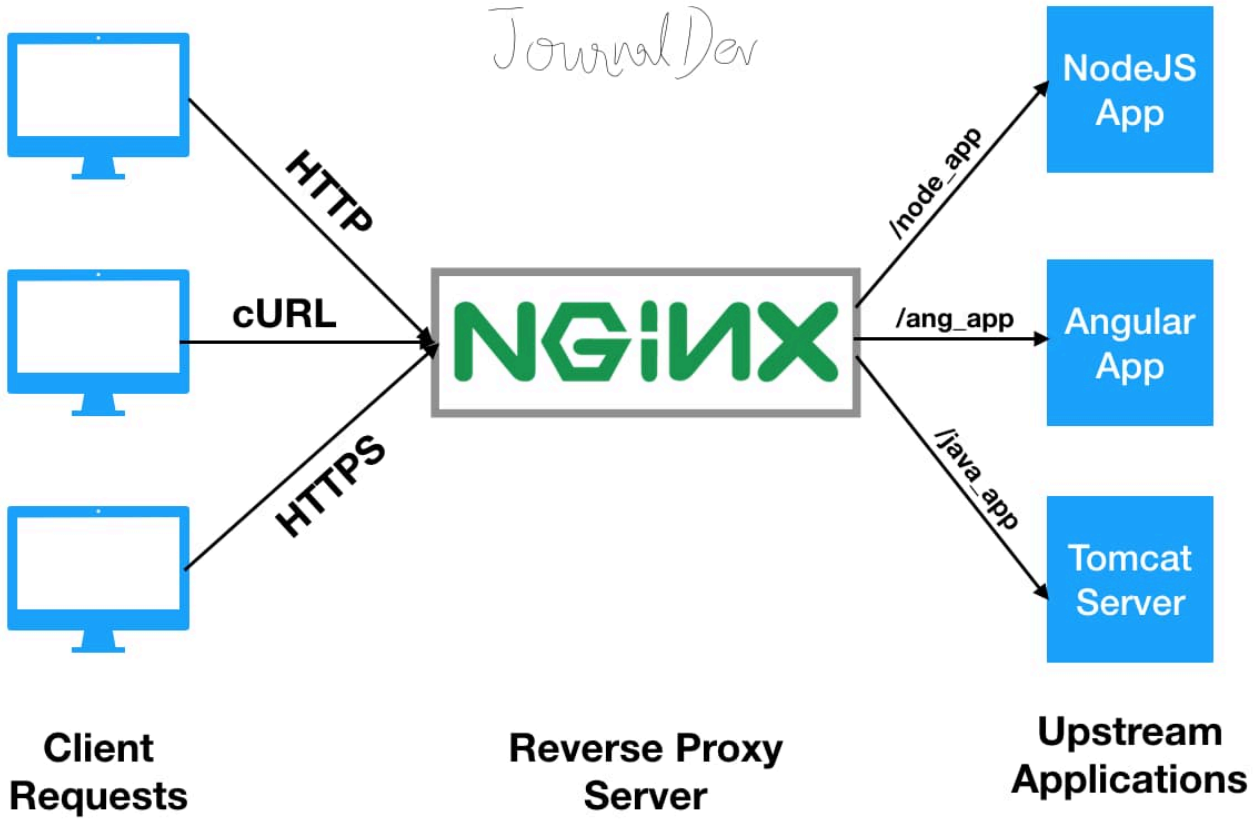
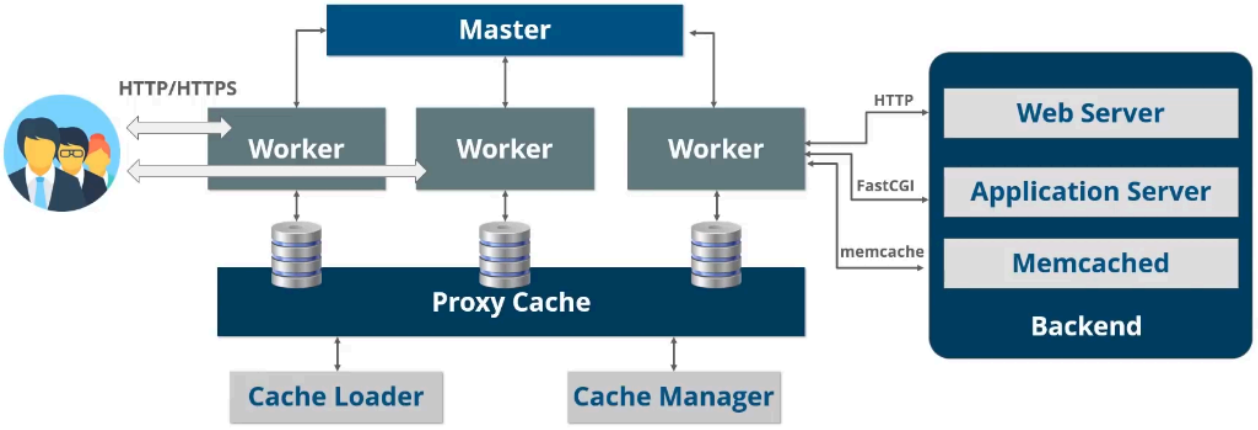
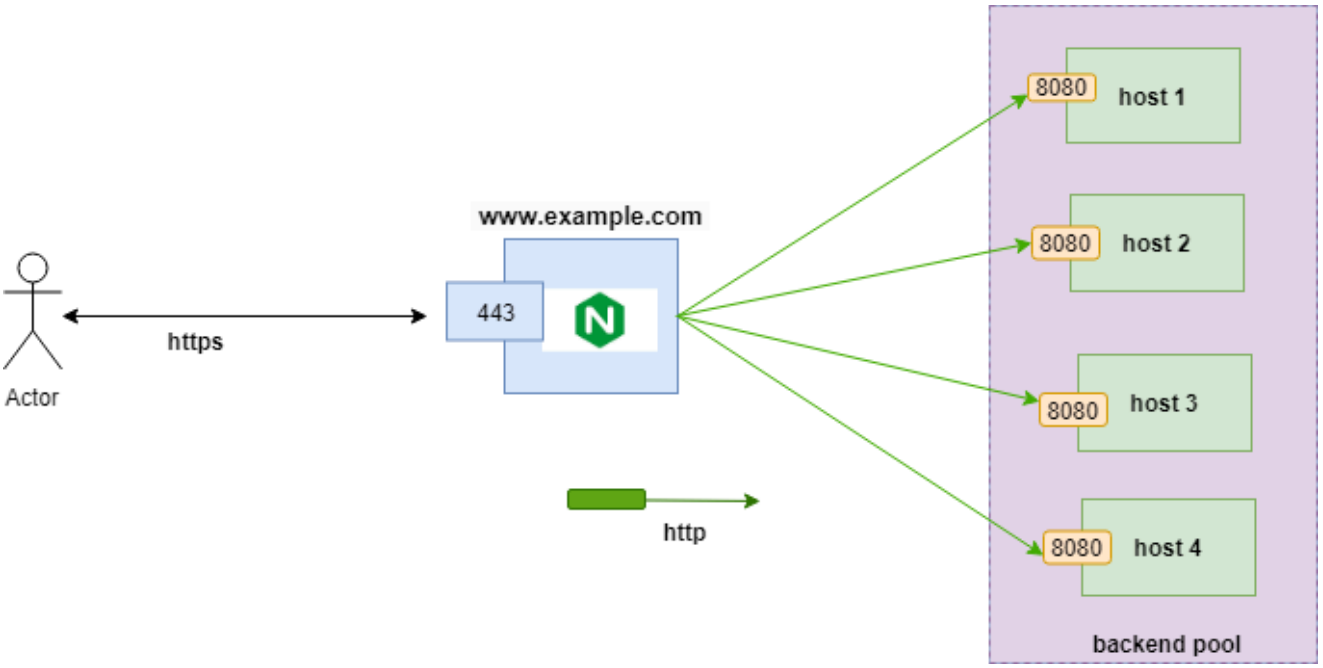
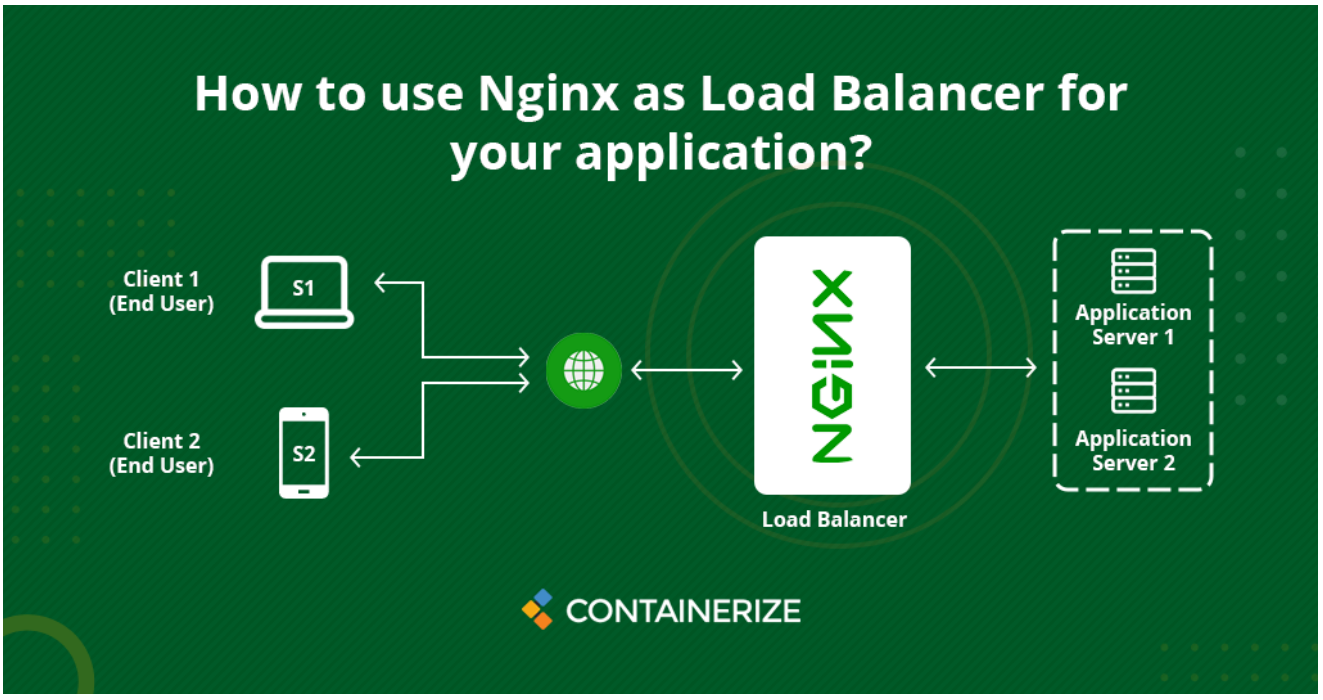
Nginx কী?


JournalDev
HTTP
cURL
HTTPS
NGINX
/node_app
/ang_app
/java_app
NodeJS App
Angular App
Tomcat Server
Client Requests
Reverse Proxy Server
Upstream Applications


Master
HTTP/HTTPS
Worker
Worker
Worker
HTTP
FastCGI
memcache
Web Server
Application Server
Memcached
Backend
Proxy Cache
Cache Loader
Cache Manager





Nginx (engine-x) হলো একটি **high-performance web server + reverse proxy + load balancer**।
এটা মূলত web traffic handle করার জন্য তৈরি, এবং আজকের বেশিরভাগ **production web applications**-এ ব্যবহার করা হয়।

1️⃣ Nginx কী কাজ করে

Nginx একসাথে কয়েকটা গুরুত্বপূর্ণ কাজ করে।

1️⃣ Web Server

- HTML, CSS, JS, image serve করে
- React/Vue build files serve করতে পারে

Example:

```
React build → Nginx → Browser
```

2️⃣ Reverse Proxy

Nginx client request backend server এ forward করে।

Example Django app:

```
User Browser
     |
     ▼
   Nginx
     |
     ▼
Gunicorn (Django)
```

User সরাসরি Django server এ connect করে না।

3️⃣ Load Balancer

Multiple backend server থাকলে traffic distribute করে।

Example:

```
          Nginx
        /   |   \
       ▼    ▼    ▼
   Django1 Django2 Django3
```

এতে site crash হওয়ার সম্ভাবনা কমে।

4️⃣ Static File Server

Django production এ static file serve করে না।

Nginx static file serve করে:

```
/static/
/media/
```

```
React build files
images
```

2️⃣ Django Production Architecture

Production এ সাধারণ architecture:

```
Browser
   |
   ▼
 Nginx
   |
   ▼
Gunicorn
   |
   ▼
Django App
   |
   ▼
PostgreSQL
```

Background tasks:

```
Django
  |
  ▼
Redis
  |
  ▼
Celery Worker
```

3️⃣ Nginx কেন দরকার

Feature	Reason
High performance	হাজার হাজার request handle করতে পারে
Static file serve	Django থেকে faster
Security	direct backend access block
Reverse proxy	backend hide করে
Load balancing	scaling support

4️⃣ Simple Nginx Config (Django)

Example:

```
server {
    listen 80;

    location / {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
    }

    location /static/ {
        alias /app/static/;
    }
}
```

Explanation:

```
/api request → Django
/static → Nginx serve
```

5️⃣ React + Django + Nginx Flow

```
User Browser
      |
      ▼
      Nginx
   /        \
React files   Django API
(index.html)  (Gunicorn)
```

Example:

```
http://site.com → React
http://site.com/api → Django
```

6️⃣ Docker Example

```
services:
  nginx:
    image: nginx
    ports:
      - "80:80"
    depends_on:
      - backend
```

7️⃣ Real Production Stack

Most professional Django apps use:

```
React
   |
   ▼
Nginx
   |
   ▼
Gunicorn
   |
   ▼
Django
   |
   ▼
PostgreSQL
   |
   ▼
Redis
   |
   ▼
Celery
```

8️⃣ Important fact

Django production **never run like this**:

```
python manage.py runserver
```

Instead:

```
Gunicorn + Nginx
```

💡 **Short summary**

Tool	Purpose
Nginx	web server + reverse proxy
Gunicorn	Django WSGI server
Redis	message broker / cache
Celery	background tasks
PostgreSQL	database

তুমি জানতে চাও **Nginx কেন ব্যবহার করি এবং না করলে কি হবে** — এটা বুঝতে হলে আগে Django/React app কিভাবে run হয় সেটা বুঝতে হবে।

1️⃣ Nginx কেন ব্যবহার করি

1️⃣ Reverse Proxy

Nginx client request backend server (Django / Node) এ forward করে।

Flow:

```
User Browser
     |
     ▼
   Nginx
     |
     ▼
 Gunicorn / Django
```

কেন দরকার

- Backend server hide করে
- security improve করে
- request filtering করতে পারে

2️⃣ Static File Serve করার জন্য

Django production এ static file serve করে না।

Example:

```
/static/css
/static/js
/images
React build files
```

Nginx এগুলো **very fast serve করে**।

Flow:

```
Browser → Nginx → static file
```

3️⃣ Performance

Nginx **event-driven architecture** ব্যবহার করে।

একসাথে হাজার হাজার request handle করতে পারে।

Example:

Server	Concurrent Request
Django runserver	~100
Gunicorn	~1000
Nginx	100k+

4️⃣ Load Balancing

Multiple backend server থাকলে Nginx traffic distribute করে।

```
        Nginx
       /  |  \
      ▼   ▼   ▼
   Django1 Django2 Django3
```

5️⃣ Security

Nginx অনেক security feature দেয়।

Example:

- Rate limiting
- DDOS protection
- IP blocking
- SSL termination

2️⃣ Nginx না ব্যবহার করলে কি হবে

1️⃣ Direct Django expose হবে

```
Browser → Django
```

Problems:

- security কম
- port expose হবে

2️⃣ Static file slow হবে

Django static serve করলে performance drop হবে।

3️⃣ High traffic handle করতে পারবে না

Gunicorn worker limited।

4️⃣ SSL handle করা কঠিন

HTTPS configure করা complicated হবে।

5️⃣ Scaling problem

Multiple server manage করা কঠিন।

3️⃣ Real Production Architecture

Production Django project সাধারণত এভাবে run করে:

```
User
  |
  ▼
Nginx
  |
  ▼
Gunicorn
  |
  ▼
Django
  |
  ▼
PostgreSQL
```

Background task:

```
Django
  |
  ▼
Redis
  |
  ▼
Celery
```

4️⃣ Development vs Production

Development

```
python manage.py runserver
```

Production

```
Nginx
   |
Gunicorn
   |
Django
```

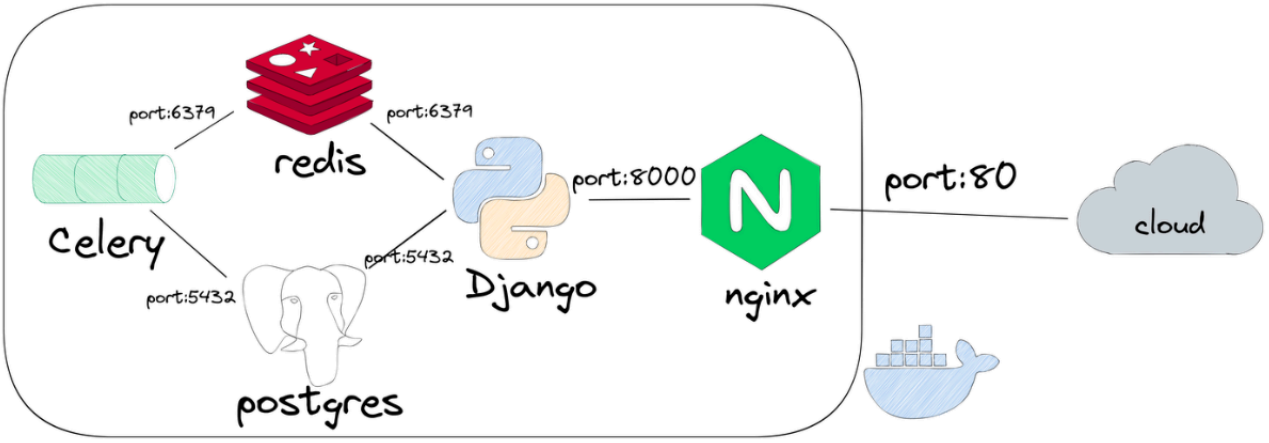
5️⃣ Short Summary

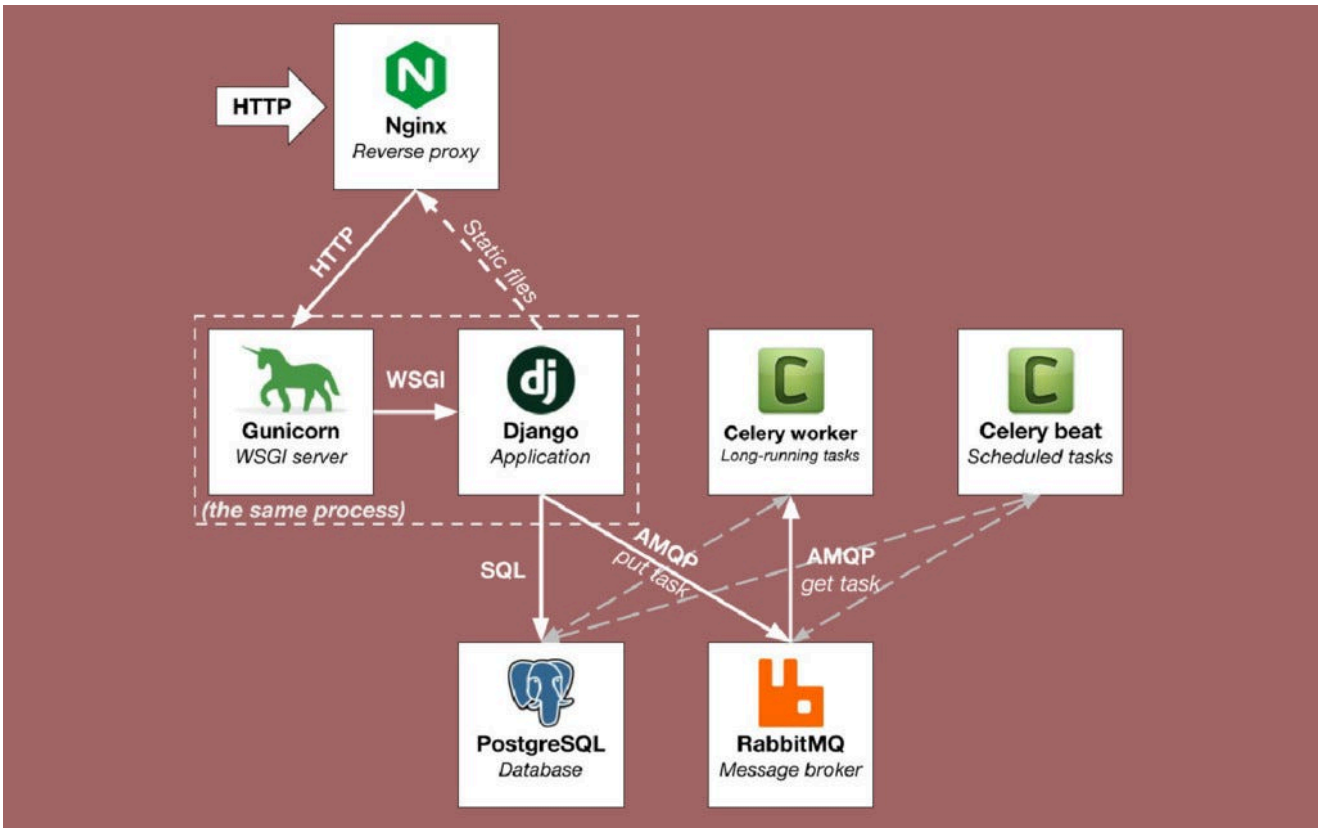
With Nginx	Without Nginx
Fast static file	slow
secure	less secure
load balancing	no
handle huge traffic	limited
SSL easy	hard

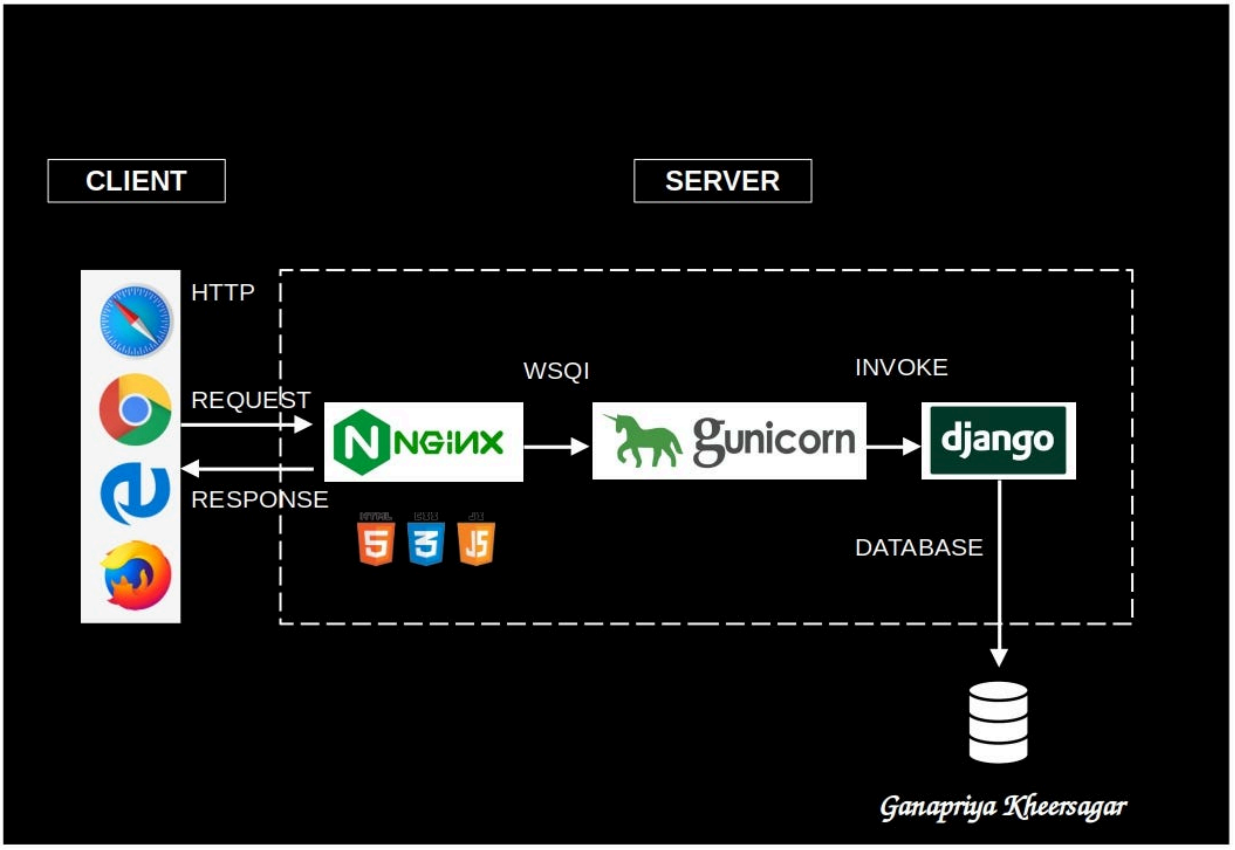
✅ **Short answer:**
Nginx হলো **fast web server + reverse proxy**।
Production web app almost সবসময় **Nginx ব্যবহার করে**।

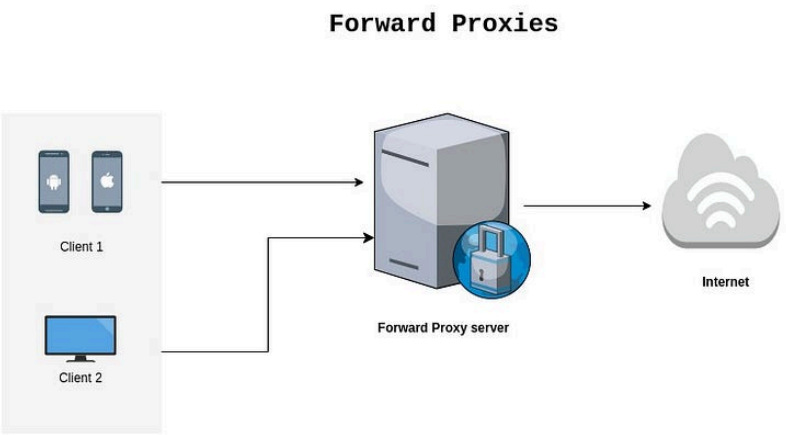
Nginx + Gunicorn + Redis + Celery + Postgres একসাথে কিভাবে কাজ করে সেটা পরিষ্কার বুঝবে।

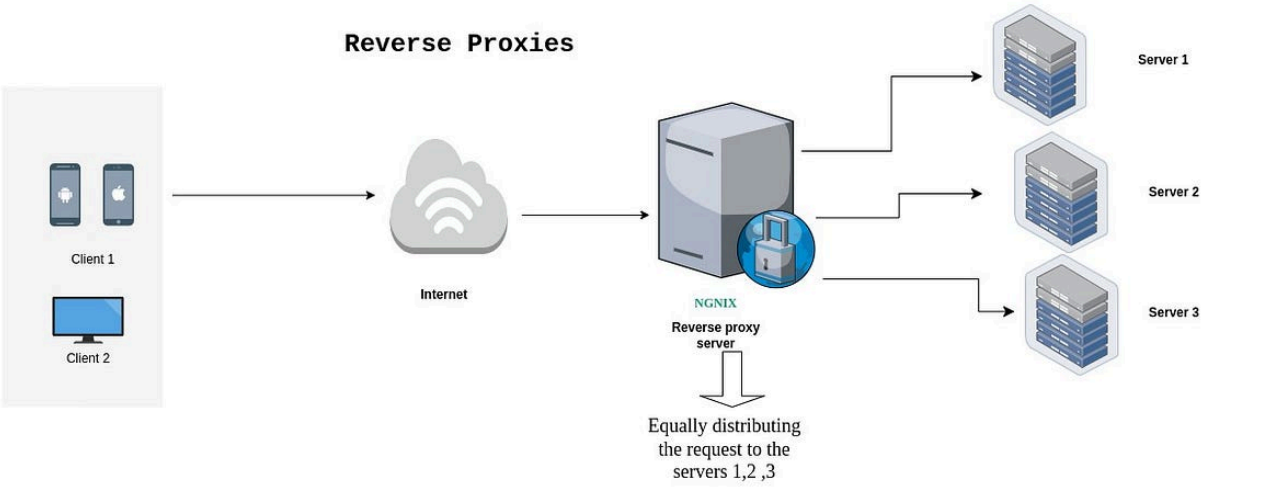
Django Production Stack: Nginx + Gunicorn + Redis + Celery + PostgreSQL




HTTP
Nginx
Reverse proxy
HTTP
Static files
Gunicorn
WSGI server
WSGI
Django
Application
(the same process)
Celery worker
Long-running tasks
Celery beat
Scheduled tasks
SQL
AMQP
put task
AMQP
get task
PostgreSQL
Database
RabbitMQ
Message broker


CLIENT
SERVER
HTTP
REQUEST
RESPONSE
WSQI
INVOKE
NGINX
gunicorn
django
DATABASE
Ganapriya Kheersagar





এখন আমরা দেখবো **এই ৫টা component একসাথে কিভাবে কাজ করে** একটি production web application এ।

1️⃣ Overall Request Flow

একটা user যখন website open করে তখন request flow সাধারণত এভাবে যায়:

```
User Browser
      |
      ▼
     Nginx
      |
      ▼
   Gunicorn
      |
      ▼
     Django
      |
      ▼
  PostgreSQL
```

Background tasks এর জন্য:

```
Django
  |
  ▼
Redis (Queue)
  |
  ▼
Celery Worker
```

2️⃣ Nginx (Entry Point)

Nginx হলো server এর প্রথম layer।

কাজ

- Reverse proxy
- Static file serve
- Load balancing
- SSL handling

Flow

```
Browser → Nginx → Backend
```

Example:

```
http://site.com → React
http://site.com/api → Django
```

3️⃣ Gunicorn (Application Server)

Gunicorn হলো Python WSGI server।

Django directly internet traffic handle করার জন্য তৈরি না।

Gunicorn Django app run করে।

```
Nginx → Gunicorn → Django
```

Gunicorn কী করে

- multiple worker process run করে

- concurrent request handle করে
- Django app serve করে

Example:

```
gunicorn config.wsgi:application --workers 3
```

4️⃣ Django (Application Logic)

Django application handle করে:

- API
- authentication
- business logic
- request processing

Example flow:

```
User login request
        |
        ▼
     Django
        |
        ▼
   PostgreSQL query
```

5️⃣ PostgreSQL (Database)

PostgreSQL হলো main database।

এখানে store হয়:

- users
- orders
- products
- logs
- application data

Flow:

```
Django → PostgreSQL
```

Example:

```
SELECT * FROM users
```

6️⃣ Redis (Message Broker / Cache)

Redis সাধারণত দুইভাবে use হয়।

1️⃣ Celery queue

```
Django → Redis → Celery Worker
```

2️⃣ Cache

```
Django → Redis cache
```

Redis RAM এ data রাখে তাই খুব fast।

7️⃣ Celery (Background Worker)

Celery background task run করে।

Example tasks:

- send email
- resize image
- generate PDF
- send notification
- ML inference

Flow:

```
Django push task
      |
      ▼
     Redis
      |
      ▼
Celery Worker execute task
```

Example:

```
send_email.delay(user.email)
```

8️⃣ Complete Architecture

সবকিছু একসাথে:

```
        User
         |
         ▼
       Nginx
         |
         ▼
      Gunicorn
         |
         ▼
        Django
       /      \
      ▼        ▼
PostgreSQL   Redis
                |
                ▼
             Celery
```

9️⃣ Real Production Example

Suppose user signup করলো।

Flow:

```
1 User signup
2 Django save user → PostgreSQL
3 Django push email task → Redis
4 Celery worker send welcome email
```

User wait করতে হয় না email sending এর জন্য।

🔟 Why this stack popular

Tool	Purpose
Nginx	web server + reverse proxy
Gunicorn	run Django app
Django	application logic
PostgreSQL	main database
Redis	cache / message broker
Celery	background task

এই stack **Netflix, Instagram, Pinterest, Shopify** টাইপ production systems এ common।

💡 যদি চাও আমি তোমাকে **beginner থেকে production level Django roadmap** বানিয়ে দিতে পারি যেখানে তুমি ধাপে ধাপে শিখবে:

```
Django
→ DRF
→ PostgreSQL
→ Redis
→ Celery
→ Docker
→ Nginx
→ Production deployment
```

এটা follow করলে তুমি **real production engineer level** Django stack শিখতে পারবে।